

Nghiên cứu Repo: mattpocock/skills

Ngày nghiên cứu: 2026-08-05

Researcher: GitHub Repo Researcher (WF-GITHUB-RESEARCH)

Nhánh: research/skills-2026-08-05

Trạng thái: Bước 3 hoàn thành — chờ user chọn Mode A / Mode B

1. Tổng quan repo

Mục	Chi tiết
Repo	[mattpocock/skills](https://github.com/mattpocock/skills)
Tác giả	Matt Pocock — người tạo Total TypeScript, ts-reset, và newsletter AI Hero (~60,000 đăng ký)
Mô tả gốc	"My agent skills that I use every day to do real engineering — not vibe coding"
Version	1.2.1 (plugin.json) / 1.2.0 (package.json)
License	MIT — tự do sử dụng, chỉnh sửa, phân phối
Stars	204,447 (cực kỳ phổ biến)
Forks	17,652
Hoạt động gần nhất	2026-08-05 (hôm nay — đang được phát triển tích cực)
Open Issues	280

Mục đích cốt lõi: Bộ skills (slash commands + behaviors) cho Claude Code và Codex được thiết kế theo tư duy kỹ thuật phần mềm thực sự — không phải "vibe coding". Thay vì kiểm soát toàn bộ quy trình như GSD/BMAD, các skill này nhỏ, có thể hack, và composable. Chúng đối phó với 4 failure mode phổ biến khi dùng AI coding agents:

1. Agent không làm đúng ý → Giải pháp: grilling session trước khi code
2. Agent quá verbose → Giải pháp: shared domain vocabulary (CONTEXT.md)
3. Code không chạy → Giải pháp: TDD + feedback loop + debug discipline
4. Codebase thành ball of mud → Giải pháp: codebase design vocabulary + architecture improvement

2. Cấu trúc repo

```
mattpocock/skills/
├── skills/
│   ├── engineering/
│   │   ├── ask-matt/
│   │   ├── grill-with-docs/
│   │   ├── triage/
│   │   ├── improve-codebase-architecture/
│   │   └── setup-matt-pocock-skills/
│   └── layout)
├── to-spec/
├── to-tickets/
└── implement/
```

← Tất cả skill definitions
← 18 skills (được promote — ship trong plugin)
← Router: chỉ user gọi, map sang các skill khác
← Grilling + domain model building (stateful)
← Issue state machine
← Setup 1 lần/repo (issue tracker, labels, doc layout)
← Conversation → spec → issue tracker
← Plan → tracer-bullet tickets với blocking edges
← Build theo spec/tickets (drives /tdd + /code-

review)	
wayfinder/	← Map huge foggy work as decision tickets
prototype/	← Throwaway HTML prototype để answer design
question	
diagnosing-bugs/	← 6-phase disciplined debug loop
research/	← Background agent investigates từ primary sources
tdd/	← Red-green-refactor TDD loop
domain-modeling/	← Sharpen project domain language → CONTEXT.md +
ADRs	
codebase-design/	← Deep module vocabulary
(module/interface/depth/seam)	
code-review/	← 2-axis parallel sub-agents review (Standards +
Spec)	
resolving-merge-conflicts/	
wizard/	← Tạo bash wizard cho bước chỉ human làm được
productivity/	← 7 skills (được promote – ship trong plugin)
grill-me/	← User-invoked grilling (stateless, không có repo)
grilling/	← Reusable interview primitive (model-invoked)
handoff/	← Compact session → file để agent mới tiếp tục
teach/	← Multi-session teaching workspace
to-questionnaire/	← Decision → questionnaire gửi người khác
wait-what/	← Re-explain message không hiểu, dùng vocab
CONTEXT.md	
writing-for-agents/	← Meta-skill: viết docs cho agents
misc/	← 4 skills (giữ lại, không promote, không ship)
in-progress/	← 6 skills (beta, public để nhận feedback, không
ship)	
deprecated/	← Skills không còn dùng
docs/	← Human-facing docs (mirrors engineering/ +
productivity/)	
engineering/	← 1 page/skill với 4 sections chuẩn
productivity/	
.agents/	← Documentation cho agent/maintainer của repo này
adr/	← Architectural decisions về system skills
install-block.md	← Cú pháp install cho readme
invocation.md	← Model-invoked vs user-invoked rules
writing-docs.md	← Template viết docs page cho skill
.claude-plugin/	← Claude Code plugin manifest
plugin.json	← Danh sách skills ship trong plugin, version
marketplace.json	← Self-hosted marketplace fallback
.out-of-scope/	← ADR-style docs: những gì đã từ chối và tại sao
scripts/	
link-skills.sh	← Symlink skills vào ~/.claude/skills,
~/.agents/skills	
list-skills.sh	
CONTEXT.md	← Shared domain glossary của chính repo skills này
CLAUDE.md	← Repo instructions cho Claude Code agents
AGENTS.md	← Symlink → CLAUDE.md (để Codex đọc cùng
instructions)	

Mỗi skill trong `engineering/` và `productivity/` có:

- **SKILL.md** — nội dung skill, frontmatter YAML (name, description, disable-model-invocation)
- **agents/openai.yaml** — Codex metadata (display_name, short_description, policy.allow_implicit_invocation)

- File companion tùy chọn: `DEEPENING.md`, `DESIGN-IT-TWICE.md`, `PHASE-BOUNDARIES.md`, `tests.md`, `mocking.md`

3. Phân tích kỹ thuật

3.1 User-invoked vs Model-invoked — Phân tách tường minh

Đây là pattern nổi bật nhất của repo. Mỗi skill thuộc đúng 1 trong 2 loại:

User-invoked (chỉ human gọi, không agent tự gọi):

```
# SKILL.md frontmatter
disable-model-invocation: true

# agents/openai.yaml
policy:
  allow_implicit_invocation: false
```

Ví dụ: `ask-matt`, `grill-with-docs`, `grill-me`, `handoff`, `implement`, `wayfinder`, `to-spec`, `to-tickets`

Model-invoked (agent hoặc human đều có thể gọi):

```
# Không có disable-model-invocation, description dùng trigger phrasing
description: "Grill the user relentlessly about a plan, decision, or idea. Use when the
user wants to stress-test their thinking, or uses any 'grill' trigger phrases."
```

Ví dụ: `grilling`, `tdd`, `diagnosing-bugs`, `codebase-design`, `domain-modeling`, `code-review`, `research`

Quy tắc thành phần: User-invoked skill có thể gọi model-invoked skill, nhưng KHÔNG BAO GIỜ gọi user-invoked skill khác. Model-invoked skill hoàn toàn độc lập.

Lý do thiết kế: Ngăn agent tự ý khởi động orchestration flow (như `grill-with-docs` hoặc `implement`) — những flow này chỉ có ý nghĩa khi human chủ động muốn. Model-invoked skills là "primitive" được tái dùng bởi nhiều user-invoked skills.

3.2 Grilling — Reusable Interview Primitive

`grilling` (model-invoked) là primitive được dùng bởi ít nhất 5 skills:

- `grill-me` (wrapper stateless)
- `grill-with-docs` (wrapper với `CONTEXT.md` + ADRs)
- `triage` (internal)
- `wayfinder` (internal)
- `improve-codebase-architecture` (internal)

Cơ chế: (`skills/productivity/grilling/SKILL.md`)

- Map decisions như **design tree**: mỗi quyết định branch thành các quyết định phụ thuộc
- Làm việc theo **rounds**: frontier = tất cả quyết định chưa settled có đủ prerequisite
- Mỗi round: hỏi toàn bộ frontier cùng lúc (đánh số + kèm recommended answer), chờ trả lời
- Agent tự tìm **facts** từ environment (sub-agent nếu cần) — không hỏi user về thứ agent tra được
- Session kết thúc khi frontier rỗng: mọi branch của design tree đã resolved

Format câu hỏi chuẩn:

```
Q1 - <question title>: <question body>
➡ <your recommended answer>
```

3.3 CONTEXT.md — Shared Domain Glossary

Pattern: mỗi repo dùng skills này nên có `CONTEXT.md` riêng, do `/grill-with-docs` và `/domain-modeling` sinh ra và maintain.

`CONTEXT.md` trong repo `mattpocock/skills` chính là ví dụ minh họa (`CONTEXT.md` tại root):

```
## Language
**Issue tracker**: The tool that hosts a repo's issues...
  _Avoid_: backlog manager, backlog backend, issue host

**Issue**: A single tracked unit of work...
  _Avoid_: ticket (chỉ dùng khi quote external system)
```

Mục đích:

- Tất cả agents cùng dùng vocabulary này thay vì phải "suy đoán jargon"
- Giảm verbosity: "problem with materialization cascade" thay vì "problem when a lesson inside a section is made real"
- File này được đọc bởi `tdd`, `diagnosing-bugs`, `code-review` để dùng vocabulary đúng
- `domain-modeling` skill chủ động challenge và update file này

3.4 Diagnosing-bugs — 6-Phase Disciplined Debug Loop

(`skills/engineering/diagnosing-bugs/SKILL.md`)

Phase 1 — Build a feedback loop (critical, tốn effort nhiều nhất):

- Phải có **tight feedback loop** trước: 1 command có thể go red on this bug
- 10 cách build: failing test, curl script, CLI invocation, headless browser, trace replay, throwaway harness, property/fuzz, bisection, differential, HITL bash script
- **Completion criterion**: "No Phase 2 if no tight loop." Nếu bắt đầu đọc code để suy ra hypothesis trước khi có loop → STOP.

Phase 2 — Reproduce + minimise

Phase 3 — Hypothesise (3-5 ranked hypotheses, falsifiable predictions)

Phase 4 — Instrument (1 variable at a time, tagged debug logs [`DEBUG-a4f2`])

Phase 5 — Fix + regression test (write test BEFORE fix, chỉ khi có correct seam)

Phase 6 — Cleanup + post-mortem (grep debug tags, thêm architectural finding)

3.5 Code-review — 2-Axis Parallel Sub-agents

(`skills/engineering/code-review/SKILL.md`)

Hai trục hoàn toàn độc lập, chạy song song:

- **Standards**: code có theo documented coding standards không? + Fowler code smells baseline
- **Spec**: code có faithfully implement originating issue/spec không?

```
Spawn cả 2 sub-agents cùng 1 lúc với Agent tool →
collect reports → aggregate dưới 2 headings riêng biệt → KHÔNG merge/rerank
```

Lý do 2 trục riêng biệt: Code có thể pass Standards nhưng fail Spec (đúng convention, sai yêu cầu), hoặc ngược lại. Trộn 2 trục khiến 1 trục mask trục kia.

Fowler smell baseline cố định (12 smells, luôn áp dụng dù repo không có coding standards): Mysterious Name, Duplicated Code, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Shotgun Surgery, Divergent Change, Speculative Generality, Message Chains, Middle Man, Refused Bequest.

3.6 Codebase-design — Deep Module Vocabulary

([skills/engineering/codebase-design/SKILL.md](#))

Định nghĩa chính xác 6 khái niệm (dùng đúng từ, không thay thế bằng "component/API/boundary"):

- **Module**: anything with interface + implementation (scale-agnostic: function, class, package, tier-slice)
- **Interface**: type signature + invariants + ordering constraints + error modes + config + perf characteristics
- **Depth**: leverage-as-interface — amount of behavior per unit of interface learned
- **Seam** (Michael Feathers): where you can alter behavior without editing in that place
- **Adapter**: concrete thing satisfying an interface at a seam (role, not substance)
- **Leverage/Locality**: what callers/maintainers get from depth

Test thực dụng:

- Deletion test: xóa module đi thì complexity biến mất (pass-through) hay xuất hiện khắp N callers (earning keep)?
- One adapter = hypothetical seam. Two adapters = real seam. Không tạo seam khi chưa cần.

3.7 Wayfinder — Map Huge Work as Decision Tickets

([skills/engineering/wayfinder/SKILL.md](#))

Dành cho work quá lớn cho 1 session, chưa rõ đường đi (foggy). Tạo **shared map** trên issue tracker.

Key distinctions:

- Decision tickets = questions, NOT implementation tasks
- 1 ticket/session maximum (trừ research tickets)
- Fog of war: chỉ chart những gì đã có thể specify — "Not yet specified" section cho fuzzy areas
- Out of scope: explicit section, không bao giờ graduate thành ticket

4 ticket types: Research (AFK, /research subagent), Prototype (HITL, /prototype), Grilling (HITL, default), Task (HITL hoặc AFK, unblocks a decision)

3.8 Writing-for-agents — Meta-skill về Document Design

([skills/productivity/writing-for-agents/SKILL.md](#))

Rất quan trọng: meta-skill viết skills/CLAUDE.md/AGENTS.md. Các concept chính:

Context pointers: Wording quyết định khi nào agent reach tài liệu — pointer yếu = variance bug.

Information hierarchy (4 tầng):

1. In-file step (primary tier — what agent does, in order)
2. In-file reference (consulted on demand — rules, facts)
3. Disclosed reference (separate file, reached by pointer, loaded only when pointer fires)

Leading words: Compact concepts từ pretraining: "tight" (loop), "red" (test state), "fog of war", "tracer bullets". Dùng token duy nhất thay vì 20 từ giải thích. Anchor: nếu word cùng xuất hiện trong prompts, docs, và codebase → agent link được.

Pruning disciplines: Single source of truth, environment là source (đừng restate), relevance check từng dòng, tránh sediment (stale layers không ai dám xóa).

Completion criteria: Phải checkable (agent tự biết done/not-done) + exhaustive (drives enough legwork).

3.9 Bucket Lifecycle + ADR trong .agents/

Skills có vòng đời rõ ràng: `in-progress/` → `engineering/` hoặc `productivity/` → có thể → `deprecated/`. Skills trong `misc/` tồn tại song song nhưng không promote.

`.agents/adr/` chứa ADR của chính hệ thống skills (không phải sản phẩm):

- `0001-explicit-setup-pointer-only-for-hard-dependencies.md` — hard vs soft dependency distinction
- `0002-ship-as-a-claude-code-plugin.md` — lý do chọn Claude plugin, roadmap Codex plugin
- `.out-of-scope/` — pattern rất hiếm: docs tường minh ghi lại những gì đã từ chối:
- `question-limits.md` — tại sao không giới hạn số câu grilling (mọi request liên quan đã được từ chối)
- `mainstream-issue-trackers-only.md`
- `setup-skill-verify-mode.md`

4. Điểm mạnh và điểm yếu

Điểm mạnh

Điểm mạnh	Mô tả
-----	-----
Composable, không monolithic	Mỗi skill nhỏ, tự chứa. Có thể chạy riêng hoặc kết hợp (implement gọi tdd + code-review)
User-invoked vs Model-invoked tường minh	Ngăn agent tự khởi động orchestration flows — chỉ human mới trigger
CONTEXT.md như shared language	Giải quyết vấn đề "agent không biết jargon" theo cách có thể persist qua session
Grilling làm primitive tái dùng	1 implementation, 5+ consumers — không duplicate logic
Phase-gated debug loop	Diagnosing-bugs không cho phép nhảy phase, có completion criteria rõ ràng
Parallel sub-agents trong code-review	2 trực không "làm bẩn" context của nhau — kết quả chính xác hơn
Writing-for-agents meta-skill	Systematic framework viết docs cho agent: leading words, information hierarchy, pruning
Lifecycle skills rõ ràng	Bucket system + <code>.out-of-scope/</code> + ADRs = biết tại sao mọi quyết định được đưa ra
MIT license + 204K+ stars	Rất trưởng thành, cộng đồng lớn, active maintenance

Điểm yếu / Giới hạn

Điểm yếu	Mô tả
-----	-----
Phụ thuộc vào issue tracker	Nhiều skills (to-tickets, triage, wayfinder) cần setup issue tracker trước — overhead với project nhỏ
Không có role-based agents	Không có khái niệm PM, TL, QA — skills thiên về "how to do", không về "who does what"

Setup 1 lần per repo	/setup-matt-pocock-skills phải chạy trước các engineering skills mới hoạt động đúng
Chưa có Codex native plugin	Đang là roadmap (xem ADR-0002) — hiện chỉ Claude Code plugin officially
In-progress skills không ship	6 skills beta chưa available trong plugin — phải cài manual qua npx

5. Hiện trạng KZTEK

5.1 Hệ thống Commands/Skills hiện có

``.claude/commands/`` — 17 skills/commands:

- `scope-check.md` — chốt workflow/priority/P0-P3 (hỏi tối đa 5 câu)
- `security-audit-stride.md` — OWASP + STRIDE audit
- `ship.md` — GO/NO-GO gate trước deploy
- `verify-pr.md` — automated PR verification
- `detect-impact.md` — phân tích blast radius khi thay đổi code
- `pre-coding-check.md` — đọc CODE-GRAPH + lessons + GOTCHAS trước khi code
- `writing-agent-skill.md` — viết skill/agent mới
- `kztek-brand-info.md` — brand guidelines
- `ui-ux-pro-max.md`, `design-taste-frontend.md` — UI/UX skills
- `graphify.md`, `run-plan-step.md`, `sync-global.md`, v.v.

``.claude/agents/`` — 19 agents theo vai trò:

CTO, Engineering Manager, Product Manager, Business Analyst, Tech Lead, Senior Developer, Junior Developer, QA Lead, QA Engineer, DevOps Lead, DevOps Engineer, Project Manager, Documentation Writer, UI/UX Designer, UX/UI Reviewer, Code Migrator, GitHub Repo Researcher, Task Planner, MD Optimizer.

5.2 So sánh trực tiếp theo từng pattern

Pattern	mattpocock/skills	KZTEK hiện tại
-----	-----	-----
Phân tách user-invoked vs model-invoked	Tường minh qua <code>disable-model-invocation: true</code> + <code>openai.yaml</code> policy	Không có — tất cả commands có thể bị agent tự kéo vào
Shared domain glossary	<code>CONTEXT.md</code> per repo, được <code>domain-modeling</code> skill maintain	Không có — KZTEK dùng các term trong <code>CLAUDE.md</code> nhưng không có glossary tập trung per-project
Grilling/interview primitive	<code>grilling</code> skill tái dùng bởi 5+ skills, có frontier/round/design-tree model	<code>scope-check.md</code> hỏi tối đa 5 câu cứng nhắc, không có design tree, không tái dùng được
Disciplined debug loop	<code>diagnosing-bugs</code> với 6 phases, completion criteria rõ ràng, không cho nhảy phase	§9a Agent Introspection (loop detection) — khác hoàn toàn về mục đích: phát hiện agent stuck, không phải debug app
TDD skill	<code>tdd</code> — red-green-refactor, seam-based, anti-patterns rõ	Không có
Code review skill	<code>code-review</code> — 2 trục (Standards +	<code>verify-pr.md</code> — kiểm tra

	Spec), parallel sub-agents	automated, không phải 2-axis review
Session handoff	<code>handoff</code> — compact session → portable markdown	Plan files (<code>PLAN-MASTER.md</code> + step files + Handoff Payload 3-key) — khác: plan-centric thay vì session-centric
Domain vocabulary	<code>codebase-design</code> — module/interface/depth/seam/adapters (6 terms chính xác)	<code>detect-impact.md</code> — tập trung blast radius, không có vocabulary layer
Huge work planning	<code>wayfinder</code> — decision tickets, fog of war, frontier model	<code>task-planner</code> + Plan file — tập trung vào task breakdown, không phải decision tree
Writing docs for agents	<code>writing-for-agents</code> — context pointers, information hierarchy, leading words, pruning	<code>writing-agent-skill.md</code> — hướng dẫn format, ít framework lý thuyết hơn
Skill lifecycle	Bucket system: in-progress → engineering/productivity → deprecated	Không có lifecycle — commands ở flat level, không có trạng thái "in-progress" hay "deprecated"
ADRs về system	<code>.agents/adr/</code> — lý do thiết kế hệ thống skill	<code>CHANGELOG-AGENTS.md</code> + <code>docs/CHANGELOG-AGENTS.md</code> — ghi lịch sử thay đổi nhưng không phải ADR format
Out-of-scope docs	<code>.out-of-scope/</code> — explicit rejection records	Không có
Plugin distribution	Claude Code plugin marketplace	Không applicable — repo cấu hình private
Prototype skill	<code>prototype</code> — throwaway HTML để trả lời design question	Không có
Router skill	<code>ask-matt</code> — map toàn bộ skills với reasoning về khi nào dùng cái gì	Không có — người dùng phải tự biết từng command
Research skill	<code>research</code> — background agent, primary sources, cited markdown	<code>github-repo-researcher</code> — tập trung vào external repos, không phải research question tổng quát

6. Thông tin repo

Mục	Chi tiết
-----	-----
URL	https://github.com/mattpocock/skills
License	MIT
Version	1.2.1
Stars	204,447
Forks	17,652
Hoạt động	Cực kỳ active — push mới nhất ngay hôm nay (2026-08-05)
Cài đặt	<code>claude plugins install mattpocock-skills</code> (Claude Code) hoặc <code>npx skills@latest add mattpocock/skills</code> (Codex/other)
Cộng đồng	~60,000 đăng ký newsletter AI Hero của tác giả
Language	Markdown (SKILL.md) + YAML (openai.yaml)

	frontmatter) — không có code runtime
Dependencies	Không có runtime dependencies — chỉ là text files

7. Bảng đề xuất cải tiến (Mode A — Bước 3b)

Trạng thái: Chờ user duyệt tại Bước 4. CHƯA áp dụng bất kỳ thay đổi nào.

Sau khi đối chiếu hiện trạng KZTEK với repo nguồn, chọn **5 đề xuất** có giá trị rõ ràng và effort/rủi ro hợp lý. Đã loại bỏ: CONTEXT.md (chỉ có ý nghĩa khi có product repo cụ thể), skill lifecycle (nice-to-have, không block gì), router skill (CLAUDE.md dispatcher đã xử lý).

E1 — Thêm `disable-model-invocation: true` cho commands chỉ nên human trigger

Cột	Nội dung
-----	-----
Học từ đâu	.agents/invokeation.md + frontmatter disable-model-invocation: true trong SKILL.md của các user-invoked skills (ask-matt, grill-with-docs, implement, wayfinder, handoff). Pattern: nếu skill orchestrate luồng lớn hoặc cần human intent rõ ràng → phải tường minh block model khỏi tự trigger.
Hiện trạng KZTEK	Tất cả 17 commands trong .claude/commands/ đều không có frontmatter disable-model-invocation. Không có cơ chế nào ngăn agent tự trigger scope-check, ship, run-plan-step, sync-global, sdk-crossplatform-eval — các commands có side effect lớn hoặc chỉ có ý nghĩa khi human chủ động.
Lý do thay đổi	Nếu không block, agent có thể tự trigger ship (deploy gate) hoặc run-plan-step (chạy bước kế tiếp trong plan) mà không có human intent — vi phạm Two-Eyes Principle ngầm. Hiện trạng này là "may mắn chưa xảy ra vấn đề" chứ không phải "đã được bảo vệ".
Áp dụng vào đâu	Thêm disable-model-invocation: true vào frontmatter của 5 commands: scope-check.md, ship.md, run-plan-step.md, sync-global.md, sdk-crossplatform-eval.md. Các commands có thể agent tự trigger hợp lý (pre-coding-check, verify-pr, detect-impact) thì giữ nguyên.
Đạt được gì	Agent không còn tự ý trigger các flow orchestration-level. Ví dụ cụ thể: trong 1 session dài, agent không tự gọi ship để deploy khi thấy code đã "xong" theo nhận định của nó — thay vào đó phải chờ human /ship.
Rủi ro/Effort	Rủi ro: Gần như không có — chỉ thêm 1 dòng frontmatter, không thay đổi behavior của skill. Effort:

E2 — Tạo skill `grilling.md` — Interview primitive tái dùng được

Cột	Nội dung
-----	-----
Học từ đâu	<code>skills/productivity/grilling/SKILL.md</code> — model interview như design tree: rounds, frontier (tất cả quyết định đã settled prerequisite), agent tự tìm facts (không hỏi user về thứ tra được), kết thúc khi frontier rỗng.
Hiện trạng KZTEK	<code>.claude/commands/scope-check.md</code> — hỏi tối đa 5 câu cứng nhắc về scope/priority/workflow, không có design tree, không có round model, không tái dùng được bởi skill khác. Khi task phức tạp cần làm rõ yêu cầu sâu hơn (VD: feature mới có nhiều nhánh quyết định), scope-check dừng sau 5 câu dù design tree chưa settled.
Lý do thay đổi	scope-check giải quyết vấn đề khác (<code>map yêu cầu → workflow ID + priority</code>) — không phải "làm rõ yêu cầu sâu". Hiện KZTEK không có primitive nào để agent gọi khi cần interview user cho đến khi "every branch of the decision tree is resolved". Nếu thiếu grilling, agent tiếp tục với hiểu biết chưa đầy đủ → sai scope → làm lại.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/grilling.md</code> . Cập nhật <code>scope-check.md</code> để gọi grilling khi scope phức tạp (thay vì tự hỏi 5 câu cứng). Cập nhật <code>CLAUDE.md</code> §Pre-0a để mention grilling là option khi scope-check không đủ.
Đạt được gì	Khi PM/Tech Lead cần làm rõ yêu cầu feature mới phức tạp, agent có thể chạy grilling session theo rounds (hỏi toàn bộ frontier, chờ answer, recompute frontier) thay vì hỏi 5 câu rồi đoán phần còn lại. Giảm vòng lặp "làm xong rồi sai yêu cầu → làm lại" — hiện tại xảy ra khi PRD dựa trên 5 câu scope-check không đủ.
Rủi ro/Effort	Rủi ro: Thấp — grilling là model-invoked (không thay thế scope-check). Effort: 2-3 giờ — viết SKILL.md theo mattpocock template, qua <code>writing-agent-skill.md</code> workflow, cập nhật 2 file.

E3 — Tạo skill `diagnosing-bugs.md` — 6-phase disciplined debug loop

Cột	Nội dung
-----	-----
Học từ đâu	<code>skills/engineering/diagnosing-bugs/SKILL.md</code> — 6 phases với completion criteria rõ ràng, đặc biệt Phase 1 "build a feedback loop" là

	critical path (10 cách build loop, không cho hypothesise khi chưa có tight loop). Keyword: "tight loop that goes red".
Hiện trạng KZTEK	CLAUDE.md §9a Agent Introspection Debugging — xử lý "agent bị stuck/loop" (≥3 retries cùng tool), không phải debugging app/feature. Không có skill nào hướng dẫn Senior Dev / QA Engineer approach bug trong sản phẩm theo discipline. Khi gặp bug khó, agent thường: đọc code → hypothesis ngay → fix thử → sai → đọc lại — không có feedback loop.
Lý do thay đổi	§9a xử lý infrastructure problem (agent stuck), không xử lý domain problem (bug trong business logic/UI). Thiếu skill debugging cho sản phẩm → agent hay bỏ qua "build feedback loop first" để hypothesise ngay (lỗi kinh điển), dẫn đến fix sai root cause hoặc mất nhiều vòng thử.
Áp dụng vào đâu	Tạo mới .claude/commands/diagnosing-bugs.md (model-invoked — agent tự trigger khi thấy user báo bug/exception). Cập nhật CLAUDE.md §4 WF-BUGFIX bước 1 để mention /diagnosing-bugs khi bug khó reproduce. Cập nhật senior-developer.md và qa-engineer.md description để trigger diagnosing-bugs khi gặp hard bug.
Đạt được gì	Senior Dev / QA gặp bug khó sẽ không hypothesise ngay mà build feedback loop trước (1 command red-capable). Cụ thể: Phase 1 completion criterion = "paste 1 command + output của nó đang red" — nếu không có thì STOP, không tiến. Giảm "fix sai root cause, phát hiện bởi QA 2 vòng sau".
Rủi ro/Effort	Rủi ro: Không có — model-invoked, không thay thế WF-BUGFIX. Effort: 2 giờ — adapt từ mattpocock (6 phases giữ nguyên logic, chỉnh ngôn ngữ + stack context C#/.NET).

E4 — Tạo skill `tdd.md` — Red-green-refactor loop

Cột	Nội dung
-----	-----
Học từ đâu	skills/engineering/tdd/SKILL.md — red-green-refactor, seam-based testing, 3 anti-patterns rõ (implementation-coupled, tautological, horizontal slicing), vertical slice model (1 test → 1 implementation → repeat).
Hiện trạng KZTEK	Không có TDD skill. verify-pr.md chạy test suite nhưng không hướng dẫn process viết test. Senior Dev / Junior Dev hiện không có guidance về "viết test trước hay code trước", không có định nghĩa "seam", không có anti-pattern checklist. CLAUDE.md §4 WF-FEATURE Bước 8-9 chỉ nói "viết unit test" không nói cách tiếp cận.
Lý do thay đổi	Không có TDD guidance → developers mặc định viết code trước, test sau (nếu có). Hệ quả đo được: tests

	thường implementation-coupled (break khi refactor dù behavior không đổi), test coverage thấp cho business logic phức tạp, QA Engineer phải catch nhiều hơn.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/tdd.md</code> (model-invoked — agent tự trigger khi user muốn build feature test-first hoặc fix bug với regression test). Cập nhật <code>WF-FEATURE Bước 8-9</code> trong <code>CLAUDE.md §4</code> để Senior/Junior Dev PHẢI dùng <code>/tdd</code> cho các phần logic phức tạp. Cập nhật <code>senior-developer.md</code> và <code>junior-developer.md</code> để mention tdd.
Đạt được gì	Developer có framework rõ ràng: seam trước, test đủ trước, code tối thiểu để pass. Anti-pattern checklist trong skill ngăn viết test tautological hoặc test implementation. <code>Verify-pr.md</code> bước 3 (test check) từ chỗ "test pass/fail" → chỗ "test đúng seam, không implementation-coupled".
Rủi ro/Effort	Rủi ro: Thấp — model-invoked, opt-in. Effort: 2-3 giờ — adapt từ mattpocock, điều chỉnh cho C#/xUnit context (thay TypeScript examples).

E5 — Tạo skill `code-review.md` — 2-axis parallel sub-agents (Standards + Spec)

Cột	Nội dung
-----	-----
Học từ đâu	<code>skills/engineering/code-review/SKILL.md</code> — 2 trục hoàn toàn độc lập, chạy song song (parallel sub-agents): (1) Standards: documented coding standards + Fowler smell baseline (12 smells); (2) Spec: faithful implementation của originating issue/spec. Merge chỉ ở output, không ở context.
Hiện trạng KZTEK	<code>verify-pr.md</code> — automated self-check (build, lint, test, security, diff). Đây là pre-PR checklist của Developer, không phải code review. <code>CLAUDE.md §4 WF-FEATURE Bước 10</code> nói "Tech Lead code review" nhưng không có skill/format nào hướng dẫn review theo chiều nào, check gì. Tech Lead review hiện dựa vào judgment cá nhân — không có checklist, không có format, không đảm bảo cả 2 trục đều được check.
Lý do thay đổi	Tech Lead review "tự do" → dễ focus 1 chiều (thường Standards) và bỏ qua Spec chiều (code đúng convention nhưng thiếu AC). Hoặc ngược lại: check spec kỹ nhưng miss code smell. 2 trục riêng biệt + parallel context đảm bảo cả 2 được check độc lập, không bias lẫn nhau. Fowler smell baseline là safety net khi repo chưa có documented standards.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/code-review.md</code> (model-invoked). Cập nhật <code>CLAUDE.md §4 WF-FEATURE Bước 10</code> và <code>WF-BUGFIX Bước 3</code> , <code>WF-</code>

	REVIEW-STD, WF-REVIEW-CRIT để Tech Lead chạy /code-review thay vì review "tự do". Cập nhật tech-lead.md agent để mention skill này.
Đạt được gì	Mỗi PR được review theo 2 trục rõ ràng: Tech Lead có output riêng biệt cho Standards vs Spec, không bỏ sót. Fowler smell baseline là backstop tối thiểu dù project chưa có CODING_STANDARDS.md. Format chuẩn giúp Developer hiểu lý do change request (Standards? Spec?).
Rủi ro/Effort	Rủi ro: Trung bình — cần đảm bảo parallel sub-agents có đủ context (diff + standards sources + spec). Nếu project không có spec/issue rõ → Spec trục chạy degraded. Effort: 3-4 giờ — adapt từ mattpocock, thêm context cho C# stack, viết Fowler smell baseline tiếng Việt nếu cần.

Tóm tắt nhanh 5 đề xuất

#	Đề xuất	Effort	Rủi ro	Impact
---	-----	-----	-----	-----
E1	Disable-model-invocation cho 5 commands	30 phút	Rất thấp	Medium
E2	Grilling primitive tái dùng	2-3 giờ	Thấp	High
E3	Diagnosing-bugs 6-phase	2 giờ	Không có	High
E4	TDD skill (red-green-refactor)	2-3 giờ	Thấp	High
E5	Code-review 2-axis parallel	3-4 giờ	Trung bình	High

Tổng effort nếu chọn cả 5: ~10-12 giờ làm việc.

Có thể chọn 0, 1, hoặc nhiều đề xuất — không bắt buộc áp dụng cả bộ.

8. Đánh giá bổ sung — Các pattern chưa đề xuất ở vòng đầu

Đánh giá thật sự "có đáng không" cho 6 pattern bị loại hoặc bỏ sót. Kết quả: 5 đề xuất mới (E6-E10), 1 pattern bị bác (Router skill), và E11 (research.md tổng quát) tách từ github-repo-researcher.

Đánh giá 1 — CONTEXT.md + domain-modeling skill

Kết luận: ĐÁNG — đề xuất E6

Trong bản đánh giá đầu tôi bác pattern này với lý do "chỉ có ý nghĩa khi có product repo cụ thể". Tuy nhiên đây là đánh giá sai ngữ cảnh:

KZTEK workspace này được dùng chung cho nhiều dự án con: iParking, iLocker, R&D, v.v. Mỗi dự án có domain terms riêng (VD: "materialization", "lane", "LPR", "bay") và hiện KHÔNG có cơ chế nào để capture và duy trì shared vocabulary đó qua các session. Agent mỗi lần phải "đoán" jargon lại từ đầu.

Một `domain-modeling.md` skill (model-invoked, đơn giản hơn phiên bản mattpocock) sẽ:

- Maintain `CONTEXT.md` cho project hiện tại khi agent detect terminology mơ hồ
- Chủ động challenge từ ngữ mâu thuẫn giữa session
- Không yêu cầu grilling (E2) — có thể hoạt động standalone

Đánh giá 2 — codebase-design vocabulary

Kết luận: ĐÁNG nhưng ưu tiên thấp — đề xuất E7

Tech Lead và Senior Dev KZTEK làm việc với C#/Avalonia/WinForms — các khái niệm "seam", "adapter", "depth" của Ousterhout áp dụng trực tiếp cho OOP systems. Giá trị chính không phải là "dạy concepts mới" mà là "đặt tên nhất quán cho những gì team đã làm nhưng không có tên chuẩn".

Cụ thể: khi TDD skill (E4) và diagnosing-bugs (E3) đều nói "test tại seam" mà không có định nghĩa seam rõ ràng → mỗi developer hiểu khác nhau → inconsistent review. `codebase-design.md` là vocabulary layer để E3/E4 hoạt động nhất quán hơn.

Phụ thuộc: E3, E4. Nên implement sau E3 và E4.

Đánh giá 3 — wayfinder vs task-planner

Kết luận: ĐÁNG nhưng effort cao — đề xuất E8

task-planner (hiện tại) giải quyết: "scope đã rõ sau scope-check → tạo kế hoạch thực thi". Nó tốt ở breakdown task đã xác định.

task-planner KHÔNG giải quyết: "chúng ta muốn xây iLocker v2 nhưng không biết bắt đầu từ đâu, kiến trúc thế nào, giao diện ra sao, có cần bỏ module X không". Đây là trường hợp điển hình khi: scope-check returns workflow nhưng task-planner không biết plan gì vì scope bản thân còn là câu hỏi.

wayfinder của mattpocock xử lý chính xác case này bằng: destination → decision tickets → resolve 1 ticket/session → fog clears dần. Phiên bản simplified cho KZTEK dùng local markdown (không cần issue tracker) hoàn toàn khả thi.

Phụ thuộc: E2 (grilling) vì mỗi decision ticket cần grilling session. Effort cao nhất trong bộ (4–5 giờ).

Đánh giá 4 — writing-for-agents → enhance KZTEK agent definitions

Kết luận: RẤT ĐÁNG, đòn bẩy cao nhất — đề xuất E9

Đây là pattern có leverage cao nhất vì ảnh hưởng toàn hệ thống, không phải 1 skill riêng lẻ.

`writing-for-agents` của mattpocock cung cấp framework có cấu trúc cho việc viết agent/skill docs. Hiện trạng KZTEK:

- `writing-agent-skill.md`: tốt về TDD-for-documentation (đưa ra process tạo mới đúng), nhưng thiếu framework về cấu trúc thông tin bên trong một definition
- `md-optimizer.md`: Phase 2 (research best practices) thiếu criteria rõ ràng để đánh giá chất lượng

- CLAUDE.md (1500+ dòng): đang có risk "sediment" (thêm vào dễ, xóa khó) và nhiều dòng có thể là no-ops với model

Các concept cụ thể từ `writing-for-agents` có thể bổ sung vào KZTEK ngay:

- **Leading words:** "tight loop", "red/green", "frontier" — KZTEK đã có một số (Two-Eyes, BLOCK, ESCALATE) nhưng chưa có framework để nhận biết và apply
- **Progressive disclosure:** quyết định in-file vs disclosed reference; hiện `writing-agent-skill.md` có mục "Bundled Resources" nhưng thiếu framework information hierarchy
- **Pruning disciplines:** single source of truth, no-op detection, staleness — giúp md-optimizer review chính xác hơn
- **Completion criteria design:** checkable + exhaustive — hiện thiếu trong nhiều agent descriptions

Đánh giá 5 — .out-of-scope/ convention

Kết luận: ĐÁNG với effort rất thấp — đề xuất E10

Hiện trạng: quyết định "không làm X" bị lost hoàn toàn sau mỗi session. LESSONS.md/GOTCHAS.md track technical issues. CHANGELOG-AGENTS.md track what changed. Không có gì track "chúng tôi đã xem xét Y và quyết định không làm vì Z".

Rủi ro: future session (hoặc future researcher) đề xuất lại cùng pattern đã bị bác, tốn thời gian xem xét lại.

Implementation đơn giản: 1 file `docs/decisions/REJECTED.md` (không cần folder, không cần skill) với format chuẩn. Effort: 30 phút để tạo file + convention.

Đánh giá 6 — ask-matt router + research.md tổng quát

Router skill: KHÔNG ĐÁNG

KZTEK đã có Dispatcher (CLAUDE.md §2 routing table) + `scope-check.md` để map yêu cầu → workflow. ask-matt giải quyết vấn đề "user không nhớ các skills nào tồn tại" — nhưng KZTEK users không gọi skills trực tiếp, họ mô tả yêu cầu và Dispatcher xử lý routing. Thêm router skill chỉ tạo thêm indirection không cần thiết.

research.md (tổng quát): ĐÁNG — đề xuất E11

Đây là pattern hoàn toàn khác với `github-repo-researcher`. `github-repo-researcher` phân tích external repos theo WF-GITHUB-RESEARCH. `research.md` của mattpocock xử lý bất kỳ câu hỏi nào cần điều tra: "library X có hỗ trợ feature Y không?", "API Z trả về format gì?", "pattern nào phù hợp nhất cho case W?".

Hiện trạng KZTEK: khi Senior Dev cần research một câu hỏi kỹ thuật, làm inline trong session chính — tốn context window, không cite sources, không có artifact. `research.md` (model-invoked) spin up background agent → investigate from primary sources → save cited Markdown → session chính tiếp tục. Pattern này đặc biệt hữu ích khi `senior-developer.md` hoặc `tech-lead.md` cần verify một technical decision.

Bảng đề xuất mới E6–E11

E6 — Tạo skill `domain-modeling.md` — Maintain shared domain vocabulary

Cột	Nội dung
----	-----

Học từ đâu	<code>skills/engineering/domain-modeling/SKILL.md</code> — actively challenge fuzzy terms, resolve overloaded words, update <code>CONTEXT.md</code> inline, record hard-to-reverse decisions as ADRs. Đọc <code>CONTEXT.md</code> hiện có trước mỗi session để dùng vocabulary nhất quán.
Hiện trạng KZTEK	Không có mechanism nào để capture và maintain shared vocabulary per-project. Mỗi session agent phải "đoán" domain jargon (iParking: "barrier", "lane", "LPR event"; iLocker: "cell", "reservation", "unlock sequence"). Vocabulary không nhất quán qua sessions → code naming không nhất quán, agent thường dùng 20 từ giải thích thay vì 1 term cố sẵn.
Lý do thay đổi	Không có shared vocabulary → agent verbose hơn cần thiết, naming trong code không đồng nhất, BA/PM và Tech Lead dùng từ khác nhau cho cùng khái niệm. Chỉ có <code>CONTEXT.md</code> project-level (nếu có) mới giải quyết được qua nhiều sessions — hiện không có ai tạo và maintain file này.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/domain-modeling.md</code> (model-invoked). Tạo template <code>docs/CONTEXT-template.md</code> để mỗi product project (iParking, iLocker) có thể bootstrap <code>CONTEXT.md</code> riêng. Cập nhật <code>senior-developer.md</code> và <code>tech-lead.md</code> description: nếu gặp term mơ hồ/conflicting, trigger <code>/domain-modeling</code> .
Đạt được gì	Mỗi product project có <code>CONTEXT.md</code> duy trì qua sessions: "barrier" không bao giờ bị gọi là "gate", "LPR event" không bị gọi là "camera trigger". Agent không phải giải thích lại jargon mỗi session. Naming trong code (variables, functions, classes) nhất quán với domain language.
Rủi ro/Effort	Rủi ro: Thấp — file <code>CONTEXT.md</code> chỉ là Markdown, không ảnh hưởng code. Effort: 2 giờ — viết skill + template.

E7 — Tạo skill `codebase-design.md` — Vocabulary layer cho module/seam/depth

Cột	Nội dung
----	-----
Học từ đâu	<code>skills/engineering/codebase-design/SKILL.md</code> — 6 terms chính xác: module (scale-agnostic), interface (không chỉ type signature), depth (leverage per unit of interface), seam (where behavior can be altered), adapter (role at a seam), leverage/locality. Deep module diagram, deletion test, "one adapter = hypothetical seam, two = real seam".
Hiện trạng KZTEK	Không có vocabulary layer cho thiết kế kiến trúc. <code>tdd.md</code> (E4) sẽ dùng từ "seam" nhưng không có định nghĩa chuẩn. Tech Lead review architecture không có criteria như "interface có đủ nhỏ không?", "depth có

	đủ không?". Code review bằng <code>code-review.md</code> (E5) dùng Fowler smells nhưng thiếu vocabulary về module design.
Lý do thay đổi	Khi E3 (diagnosing-bugs) nói "không có correct seam để write regression test" và E4 (TDD) nói "test only at pre-agreed seams" mà không có định nghĩa seam chung → mỗi developer hiểu khác nhau → review inconsistent. Vocabulary layer giúp E3/E4/E5 hoạt động nhất quán.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/codebase-design.md</code> (model-invoked — agent tự trigger khi cần design module interface hoặc tìm seam cho test). Cập nhật <code>tdd.md</code> (E4) để reference codebase-design khi discuss seams. Cập nhật <code>tech-lead.md</code> description.
Đạt được gì	Tech Lead và Senior Dev có chung vocabulary: "module này shallow quá" thay vì "class này làm quá ít", "seam ở đây đúng không?" thay vì "nên test class này hay class kia?". Deletion test ("xóa module đi complexity có về N callers không?") trở thành câu hỏi standard trong architecture review.
Rủi ro/Effort	Rủi ro: Thấp — vocabulary layer, không thay đổi process. Phụ thuộc: nên implement SAU E3 và E4 để có ngữ cảnh. Effort: 1-2 giờ — adapt từ mattpocock cho C#/OOP context.

E8 — Tạo skill `wayfinder-lite.md` — Map work lớn/mù mờ bằng decision tickets

Cột	Nội dung
-----	-----
Học từ đâu	<code>skills/engineering/wayfinder/SKILL.md</code> — destination → decision tickets (questions, not implementation tasks) → resolve 1 ticket/session → fog clears. Frontier = open unblocked unclaimed tickets. "Not yet specified" section cho foggy areas. 4 ticket types: Research, Prototype, Grilling, Task.
Hiện trạng KZTEK	<code>task-planner</code> agent giải quyết "scope đã rõ → create execution plan". Không có gì giải quyết "chúng ta muốn làm X lớn nhưng chưa biết kiến trúc thế nào, scope bao nhiêu, cần hỏi những câu gì" — case này xảy ra khi bắt đầu new product feature (iLocker v2, tích hợp mới) hoặc major refactor. Hiện: team thường jump thẳng vào task-planner với scope chưa rõ → plan sai → làm lại.
Lý do thay đổi	task-planner assumes scope known. Nếu dùng task-planner sớm khi scope còn foggy: plan bị sai direction, bước tạo ra không phải steps cần làm mà là questions cần trả lời. wayfinder-lite giải quyết phase "discover what to build" trước khi task-planner handle "how to build it".
Áp dụng vào đâu	Tạo mới <code>.claude/commands/wayfinder-lite.md</code> (user-invoked — user chủ động khi nhận ra

	effort quá lớn/foggy cho task-planner). Dùng local markdown (<code>.scratch/<feature>/decisions/</code>) thay vì external issue tracker. Cập nhật <code>CLAUDE.md</code> §Pre-0a: khi scope-check xác định work quá lớn → suggest wayfinder-lite trước task-planner. Cập nhật <code>task-planner.md</code>: nếu bị gọi khi scope unclear → gợi ý wayfinder-lite.
Đạt được gì	Team có framework cho "fog of war" planning: biết rõ khi nào cần "giải quyết câu hỏi" vs "viết code". Decision tickets trong <code>.scratch/</code> là artifact có thể revisit giữa sessions. Destination-first approach ngăn task-planner tạo plan sai direction.
Rủi ro/Effort	Rủi ro: Trung bình — nếu viết sai, team có thể dùng wayfinder khi không cần (overhead). Cần quy tắc rõ phân biệt với task-planner. Phụ thuộc: hoạt động tốt nhất khi E2 (grilling) đã có. Effort: 4-5 giờ — phức tạp nhất trong bộ, cần adapt wayfinder logic cho local-file tracker.

E9 — Cải thiện `writing-agent-skill.md` + `md-optimizer.md` bằng framework writing-for-agents

Cột	Nội dung
Học từ đâu	<code>skills/productivity/writing-for-agents/SKILL.md</code> — Information hierarchy (in-file step / in-file reference / disclosed reference); Leading words (compact concepts từ pretraining: "tight", "red", "frontier"); Context pointers (front-load trigger, one branch per trigger, cut identity); Completion criteria (checkable + exhaustive = drives legwork); Pruning (single source of truth, no-op detection, staleness check, sediment risk); Progressive disclosure.
Hiện trạng KZTEK	<code>writing-agent-skill.md</code>: tốt về TDD-for-documentation process nhưng thiếu framework cấu trúc thông tin bên trong definition — không có guidance về khi nào nên in-file reference vs disclosed reference, không có leading words concept, không có pruning criteria. <code>md-optimizer.md</code>: Phase 3 "Analyze" chỉ list ưu/nhược điểm tự do, thiếu structured criteria để đánh giá chất lượng định nghĩa (information hierarchy, no-op detection). <code>CLAUDE.md</code> 1500+ dòng: growing risk of sediment, một số dòng có thể là no-ops với model.
Lý do thay đổi	<code>writing-agent-skill.md</code> dạy "quy trình tạo skill" nhưng không dạy "skill viết tốt trông như thế nào về cấu trúc thông tin". <code>md-optimizer.md</code> tìm nhược điểm theo trực giác, không có framework đánh giá. Kết quả: agent/skill mới đúng process nhưng có thể verbose, có sediment, context pointer yếu → agent bỏ qua hoặc trigger sai lúc. Bổ sung writing-for-agents framework = cải thiện chất lượng MỌI agent/skill tạo ra từ đây,

Áp dụng vào đâu	không chỉ 1 skill cụ thể. Edit <code>writing-agent-skill.md</code> : thêm mục "Cấu trúc thông tin trong definition" (information hierarchy, leading words, pruning checklist) sau bước GREEN, trước REFACTOR. Edit <code>md-optimizer.md</code> : thêm Phase 2b "Đánh giá cấu trúc thông tin" (sau Research) với criteria cụ thể từ writing-for-agents. Không tạo file mới — tích hợp vào tools đã có.
Đạt được gì	Mọi skill/agent mới tạo ra sau này tự động pass qua writing-for-agents checklist: description front-loads trigger, no in-file reference buried under steps, leading words được dùng nhất quán (VD: "tight loop" thay vì "fast deterministic low-overhead feedback mechanism"), completion criteria rõ done/not-done. md-optimizer có thêm axis để đánh giá chất lượng: "step bị chôn bởi reference?", "có leading word nào đang được restate thành 20 từ không?".
Rủi ro/Effort	Rủi ro: Thấp — bổ sung vào files hiện có, không thay đổi behavior hiện tại của skills. Effort: 2-3 giờ — Edit 2 files + viết checklist cụ thể từ writing-for-agents concepts.

E10 — Tạo `docs/decisions/REJECTED.md` — Convention ghi lại quyết định "không làm"

Cột	Nội dung						
-----	-----						
Học từ đâu	<code>.out-of-scope/</code> folder trong mattpocock/skills — mỗi file ghi rõ: pattern/feature đã xem xét, lý do từ chối, prior requests từ cộng đồng. Pattern: explicit rejection record ngăn tái đề xuất không cần thiết. Áp dụng: không cần folder riêng, 1 file tập trung với format chuẩn là đủ.						
Hiện trạng KZTEK	Không có record nào cho quyết định "không làm". LESSONS.md/GOTCHAS.md track issues kỹ thuật. CHANGELOG-AGENTS.md track changes. RESEARCH docs ghi chú loại bỏ (VD: "Đã loại bỏ: CONTEXT.md — ...") nhưng rải rác trong nhiều RESEARCH files, không tra cứu được. Sau 8 RESEARCH sessions, nhiều patterns đã từng xem xét sẽ bị xem xét lại trong sessions tiếp theo.						
Lý do thay đổi	Không có rejection record → rủi ro "groundhog day": agent/researcher mới trong session tương lai xem xét lại cùng pattern đã từng đánh giá và quyết định không làm, tốn thời gian đánh giá lại. Bản thân research						

	session này đã lặp lại xem xét một số patterns từ các RESEARCH trước.					
Áp dụng vào đầu	Tạo docs/decisions/REJECTED.md với format: `#	Pattern	Lý do từ chối	Điều kiện mở lại	Nguồn tham khảo	Ngày`. Populate ngay với các patterns đã từng xem xét và bác trong các RESEARCH sessions hiện có (anthropics/skills, gitnexus, memmachine, ui-ux-pro-max, mattpocock/skills). Cập nhật github-repo-researcher agent description: khi loại bỏ pattern khỏi đề xuất, ghi vào REJECTED.md.
Đạt được gì	Mọi pattern đã xem xét và từ chối đều có record với lý do. Researcher tương lai tra cứu REJECTED.md trước khi đề xuất → không đề xuất lại cái đã bác. Giảm "same proposal revisited" từ trung bình ~1-2 pattern/research session xuống 0.					
Rủi ro/Effort	Rủi ro: Không có — chỉ là Markdown file. Effort: 30–45 phút — tạo file + populate 10-15 entries từ các RESEARCH sessions hiện có.					

E11 — Tạo skill `research.md` — General research skill (khác với github-repo-researcher)

Cột	Nội dung
-----	-----
Học từ đâu	skills/engineering/research/SKILL.md — spin up background agent, investigate from primary sources (official docs, source code, specs, first-party APIs — not secondary write-ups), write findings to cited Markdown file, save where repo already keeps such notes. Session chính tiếp tục trong khi background agent research.
Hiện trạng KZTEK	github-repo-researcher agent: phân tích external GitHub repos theo WF-GITHUB-RESEARCH flow phức tạp (clone, phân tích, đề xuất, merge). Không có gì để "research câu hỏi kỹ thuật tổng quát" — khi Senior Dev cần biết "Avalonia DataGrid có support virtual scrolling không?", "protocol X trả về format Y không?", họ research inline trong session chính → tốn context window, không cite sources, không có artifact để revisit.
Lý do thay đổi	Research inline = tốn context window của session

	chính (agent phải load docs, read multiple pages, summarize — tất cả trong cùng 1 context). Không cite sources = không verify được sau. Không có artifact = kiến thức mất sau session. Background agent approach của mattpocock giải phóng context window của session chính đồng thời tạo cited artifact.
Áp dụng vào đâu	Tạo mới <code>.claude/commands/research.md</code> (model-invoked — agent tự trigger khi cần investigate kỹ thuật question từ primary sources). Output: <code>docs/research/RESEARCH-[topic]-[date].md</code> với citations (khác RESEARCH-[repo] files). Cập nhật <code>tech-lead.md</code> và <code>senior-developer.md</code> description: khi cần verify technical decision từ docs, trigger <code>/research</code> thay vì <code>research</code> inline.
Đạt được gì	Session chính không bị chiếm context bởi research work. Technical findings có citations → có thể verify lại. Artifact <code>docs/research/RESEARCH-[topic]-*.md</code> có thể share giữa sessions. Senior Dev không còn "nghĩ mình nhớ docs nói gì" → phải cite source.
Rủi ro/Effort	Rủi ro: Thấp. Cần phân biệt rõ scope với github-repo-researcher (research.md: research technical questions; github-repo-researcher: analyze external repos theo WF-GITHUB-RESEARCH). Effort: 1–2 giờ — skill ngắn gọn, logic đơn giản (spin up background agent, primary sources only, cite findings).

Tổng kết bổ sung: Không đề xuất

Router skill (ask-matt analog): KHÔNG ĐÁNG. KZTEK đã có Dispatcher (CLAUDE.md §2 + CORE.md routing table) xử lý mapping yêu cầu → workflow. User KZTEK không invoke skills trực tiếp — họ mô tả yêu cầu và Dispatcher handle. Thêm router skill = indirection không cần thiết.

Tóm tắt đầy đủ E1–E11

#	Đề xuất	Effort	Rủi ro	Impact	Phụ thuộc
---	-----	-----	-----	-----	-----
E1	disable-model-invocation (5 commands)	30 phút	Rất thấp	Medium	Không
E2	grilling.md — interview primitive	2–3 giờ	Thấp	High	Không
E3	diagnosing-bugs.md — 6-phase debug	2 giờ	Không có	High	Không
E4	tdd.md — red-green-refactor	2–3 giờ	Thấp	High	Không
E5	code-review.md — 2-axis parallel	3–4 giờ	Trung bình	High	Không
E6	domain-modeling.md — project vocabulary	2 giờ	Thấp	High	Không (E2 giúp tốt hơn)
E7	codebase-design.md —	1–2 giờ	Thấp	Medium	E3, E4

	module vocabulary				
E8	wayfinder-lite.md — foggy work mapping	4–5 giờ	Trung bình	High	E2
E9	Enhance writing-agent-skill + md-optimizer	2–3 giờ	Thấp	Very High	Không
E10	docs/decisions/REJECTED.md convention	30–45 phút	Không có	Medium	Không
E11	research.md — general research skill	1–2 giờ	Thấp	High	Không

Tổng effort nếu chọn cả 11: ~22–28 giờ làm việc.

Khuyến nghị nhóm ưu tiên cao (impact cao, effort hợp lý, không phụ thuộc nhau): E1 + E3 + E9 + E10 + E11 (~7–8 giờ).

Nhóm ưu tiên trung bình (cần grilling E2 trước): E2 + E4 + E5 + E6 (~9–12 giờ).

Nhóm phụ thuộc (implement sau nhóm trên): E7 (sau E3/E4) + E8 (sau E2) (~5–7 giờ).

9. Bước 4b — Trạng thái áp dụng

User xác nhận áp dụng TOÀN BỘ E1–E11 (2026-08-05). Bước 4b đã hoàn thành trên nhánh [research/skills-2026-08-05](#).

#	Đề xuất	Trạng thái	Artifact đã tạo/sửa
---	-----	-----	-----
E1	disable-model-invocation cho 5 commands	✓ Đã áp dụng	.claude/commands/scope-check.md , ship.md , run-plan-step.md , sync-global.md , sdk-crossplatform-eval.md
E2	grilling.md — interview primitive	✓ Đã áp dụng	.claude/evals/grilling.md (mới), .claude/commands/grilling.md (mới), .claude/commands/scope-check.md (cập nhật — gợi ý grilling khi phức tạp)
E3	diagnosing-bugs.md — 6-phase debug loop	✓ Đã áp dụng	.claude/evals/diagnosing-bugs.md (mới), .claude/commands/diagnosing-bugs.md (mới)
E4	tdd.md — Red-Green-Refactor seam-based	✓ Đã áp dụng	.claude/evals/tdd.md (mới), .claude/commands/tdd.md (mới)
E5	code-review.md — 2-axis Standards + Spec	✓ Đã áp dụng	.claude/evals/code-review.md (mới), .claude/commands/code-review.md (mới)
E6	domain-modeling.md — CONTEXT.md vocabulary	✓ Đã áp dụng	.claude/evals/domain-modeling.md (mới), .claude/commands/domain-modeling.md (mới), docs/CONTEXT-template.md (mới)
E7	codebase-design.md — 7-term vocabulary	✓ Đã áp dụng	.claude/evals/codebase-design.md (mới), .claude/commands/codebase-design.md (mới)
E8	wayfinder-lite.md — foggy work decision ticket	✓ Đã áp dụng	.claude/evals/wayfinder-lite.md (mới), .claude/commands/wayfinder-lite.md (mới)

E9	Nâng cấp cấu trúc thông tin trong agent/skill definitions	✓ Đã áp dụng	<code>.claude/commands/writing-agent-skill.md</code> (thêm Bước 4b — information structure), <code>.claude/agents/md-optimizer.md</code> (thêm PHASE 2b — Information Structure Audit)
E10	<code>docs/decisions/REJECTED.md</code> — rejection convention	✓ Đã áp dụng	<code>docs/decisions/REJECTED.md</code> (mới, 10 entries), <code>docs/decisions/REJECTED.docx</code> + <code>.pdf</code>
E11	<code>research.md</code> — general research skill	✓ Đã áp dụng	<code>.claude/evals/research.md</code> (mới), <code>.claude/commands/research.md</code> (mới)

Cập nhật CLAUDE.md và agent descriptions:

- `CLAUDE.md §3.0 Pre-0a`: Thêm gợi ý grilling khi scope phức tạp sau scope-check
- `CLAUDE.md §4 WF-BUGFIX Bước 1`: Thêm mention `/diagnosing-bugs`
- `CLAUDE.md §4 WF-FEATURE Bước 8-9`: Thêm mention `/tdd`
- `.claude/agents/senior-developer.md`: Thêm section "Skills dùng trong công việc" (tdd, diagnosing-bugs, code-review, grilling, codebase-design)
- `.claude/agents/tech-lead.md`: Thêm section "Skills dùng trong công việc" (code-review, codebase-design, grilling)
- `.claude/agents/qa-engineer.md`: Thêm section "Skills dùng trong công việc" (diagnosing-bugs)

Commits: 3ea9c3a (Nhóm 1), 5c1f1c7 (Nhóm 2), a4fb3e5 (Nhóm 3)

Chờ xác nhận: Bước 5 — user xác nhận merge nhánh `research/skills-2026-08-05` về `main`.